

Undergraduate AI Capstone NYCU Spr2024 Project #2

Team6 110550170 戚維凌 110550034 孫承瑞 110550175 鄭栩安

I. Methods

1. Minimax

Minimax is a classic decision-making algorithm used in game theory, particularly in two-player deterministic games with perfect information. The goal is to determine the worst possible outcome for each alternative, and select the one with the best “worst possible outcome”.

However, in Battle Sheep, it was hard for us to find a suitable evaluation function before the game is terminal. Also, the variant rules seemed to be difficult to implement with Minimax. Therefore, we didn’t choose it.

2. Monte Carlo Tree Search (MCTS)

MCTS is a heuristic search algorithm famous for its ability to handle large search spaces. It focuses on the more promising areas, without needing specific knowledge or evaluation functions, so it’s versatile and applicable to various domains.

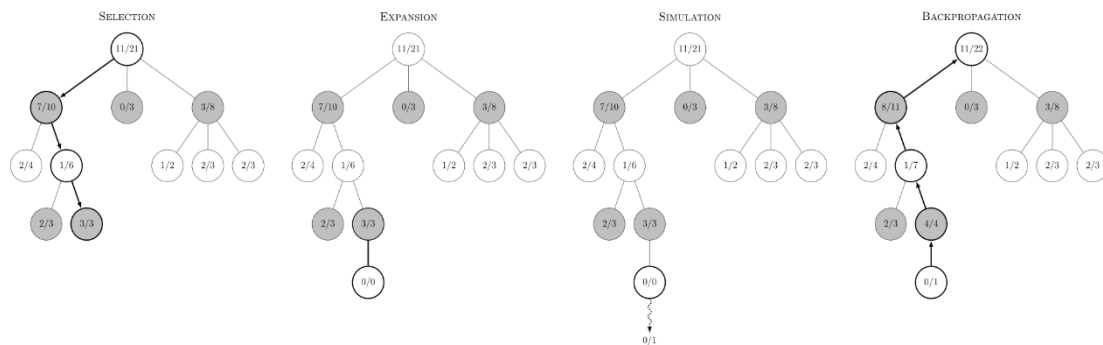


Fig.1. The steps of MCTS.

In Fig. 1., there are four steps in a simulation of MCTS:

- **Selection:**
Starting from the root node of the search tree, the algorithm traverses down the tree according to a selection policy until reaching a leaf node.
- **Expansion:**
Once a leaf node is reached, the algorithm expands it by adding one or more child nodes which representing legal moves from the current game state.
- **Rollout (Simulation):**
The algorithm chooses a newly added child node and completes a rollout from it by simply choosing random moves until the game is over.

- Backpropagation:
After the rollout, the algorithm uses the result to update the value of the nodes along the selection path.

Because of its adaptive nature, we implemented MCTS as our main algorithm.

3. AlphaZero

AlphaZero is an advanced artificial intelligence algorithm developed by DeepMind. It's famous for mastering games like chess, Go, and shogi without human knowledge. AlphaZero consists of a deep neural network that learns to evaluate game states and predict results, combined with the MCTS algorithm for searching and selecting next moves. The network is trained through self-play, where it plays against itself and learns from the outcomes.

We tried to train our own model and combine it with MCTS; however, we failed due to the time limit and the complex details.

II. Implementation Details

Agent 1:

This agent is the basic form which has 4 players, 16 sheep per player and 64-cell playing field within a 12x12 square.

We implemented **multi-threaded MCTS** to achieve parallelization. In Fig. 2., there are different types of parallelization of MCTS, and we chose the “root parallelization” since it is effective and easy to implement.

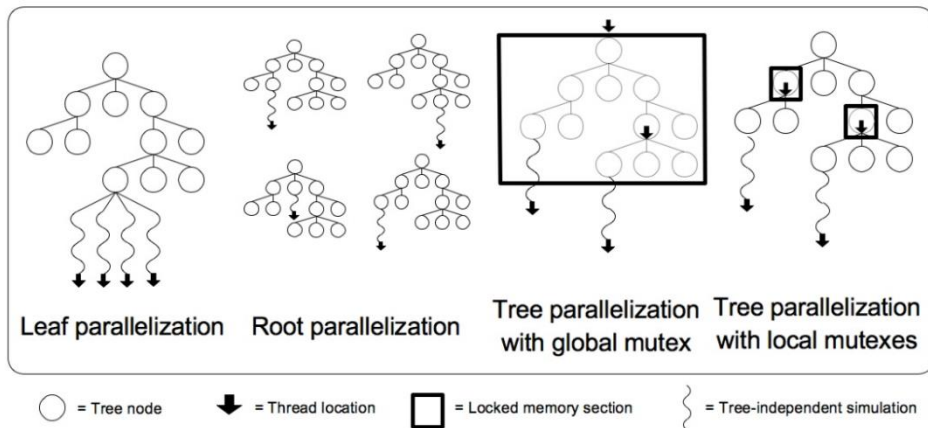


Fig. 2. Four types of parallelization of MCTS

We build multiple MCTS trees in parallel, with one thread per tree. Also, we set the max simulation time to **2500 milliseconds**. When all the simulations are done, the child nodes of root in separate trees will be merged by averaging their values, and the

best move is selected based on the result. Since the threads don't share information with each other, this method doesn't require any mutex, making the parallelization more effective.

In the selection step of MCTS, we use **UCT (Upper Confidence Bounds to Trees) formula** as the selection policy because it considers both the estimated value and the uncertainty of the node to balance between exploration and exploitation.

$$\frac{W_i}{n_i} + c \sqrt{\frac{\ln N_i}{n_i}}$$

W_i is the total score for the node considered after the i_{th} move,
 n_i is the number of simulations for the node considered after the i_{th} move,
 N_i is the total number of simulations for the parent node after the i_{th} move,
and c is the exploration parameter, theoretically equal to $\sqrt{2}$.

Equation 1. The UCT formula.

In the backpropagation step, we calculate the score for each player at the terminal game state and then compute the average score for each node as the value.

$$\text{The score of a player} = \text{round}[\text{SUM}((\text{size of each connected region of a player's cells})^{1.25})]$$

Equation 2. The score calculation.

Agent 2:

This agent is played on the overcrowded bigger board which has 4 players, 32 sheep per player and 100-cell playing field within a 15x15 square.

We use the same architecture as Agent 1, only altering the setting of number of sheep and the size of board.

Agent 3:

This agent has the same form as Agent 1 with imperfect information: A player does not know the numbers of sheep in the cells occupied by the other players.

We use the similar architecture to Agent 1. However, due to the imperfect information, we consider the number of sheep in the cells occupied by the other players as the **maximum possible number**. For example, if Player 2 has occupied two cells, then the number of sheep in each cell will be assumed as **(the total number of sheep – the number of occupied cells + 1) = 16 – 2 + 1 = 15**. Therefore, the number of legal moves for the other players will increase.

Agent 4:

This agent has the same form as Agent 1 with cooperation. The four players form two pairs: Players 1 & 3, and Players 2 & 4.

We use the similar architecture to Agent 1, but in the backpropagation step, we

calculate the sum of score for players in the same pair instead. By doing so, the player will choose the best move that can increase not only his own score but also his partner's.

III. Experiments

In this section, we indicate the experiments that include comparison with three aspects in four games,

1. Parallel and non-parallel
2. The agent being a first player and being a last player
3. Different strategies

Game 1

- Parallel and non-parallel:

In this session, we all use Agent1 as player 4 to conduct the experiment since the order of players in this battle sheep game has large impact on results. Table 1 shows the result of win rate and average score of 20 games that our Agent 1 with non-parallel MCTS and parallel MCTS respectively play against the 3 sample agents.

Method	Win Rate	Average Score
Non-parallel MCTS	95%	28.298
Parallel MCTS w/ 5 threads	100%	30.089

Table 1. The result from parallel and non-parallel MCTS agent in Game 1

It is obviously that the decision of the agent which is use parallel MCTS with 5 threads is more precisely because it makes decision that relies on combining 5 independent results.

- The agent being a first player and being a last player:

In this experiment, we all use parallel MCTS on Agent 2. As we mentioned in last session, the order of players in this battle sheep game has large impact on results. Therefore, we conducted another experiment with different order of players. Similarly, Table 2 indicates that the result of 20 games that our Agent 1 being first player and last player respectively play against the 3 sample agents. Besides, table 3 also indicates the result that those two agents play together for 20 games.

Method	Win Rate	Average Score
First Player	100%	29.881
Last Player	100%	30.089

Table 2. The result from parallel MCTS agent being first player and being last player in Game 1

Method	Win Rate	Average Score
First Player (Against together)	80%	30.470
Last Player (Against together)	20%	29.734

Table 3. The result from parallel MCTS agent being first player and being last player together in Game 1

According to the result, it seems that though table 2 cannot show the different performance between being first player and being last player due to statistical error, table 3 indicate that the first player is able to restrict the performance of being last player significantly.

- Different strategy:

Agent 1 doesn't have many effective strategies worth conducting experiment. Thus, we will skip this session.

Game 2

- Parallel and non-parallel:

The situation is the same as Game 1 except that 32 sheep per player and 100-cell playing field within a 15x15 square for 20 games. We directly show the results.

Method	Win Rate	Average Score
Non-parallel MCTS	100%	64.463719
Parallel MCTS w/ 5 threads	100%	64.7078275

Table 4. The result from parallel and non-parallel MCTS agent in Game 2

Since the number of sheep increases compared to Game 1, the number of possible sub-nodes of each state also increases, which may result in the smaller differences between the agents with parallel and non-parallel MCTS.

- The agent being a first player and being a last player:

We conducted the similar experiments in Game 2. Following tables show the result of our Agent 2 being first player and last player respectively play against the 3 sample agents and those two agents playing together for 20 games with parallel MCTS.

Method	Win Rate	Average Score
First Player	100%	68.83016
Last Player	100%	64.7078275

Table 5. The result from parallel MCTS agent being first player and being last player in Game 2

Method	Win Rate	Average Score
First Player (Against together)	50%	63.273
Last Player (Against together)	50%	69.208

Table 6. The result from parallel MCTS agent being first player and being last player together in Game 2

Because the condition in Game 2 is more complicate, the impact from the order of player is not quite large.

- Different strategy:
Agent 2 is much like Agent 1. Thus, we still skip this session.

Game 3

- Parallel and non-parallel:
We conducted the same experiment from Game1 in this session.

Method	Win Rate	Average Score
Non-parallel MCTS	100%	26.479
Parallel MCTS w/ 5 threads	100%	26.793

Table 7. The result from parallel and non-parallel MCTS agent in Game 3

The result is the same as Game 2 in this session. As a result of our implementation in Game 3, the number of possible sub-nodes of each state also increases, making the slight differences between the agents with parallel and non-parallel MCTS.

- The agent being a first player and being a last player:
In this experiment, we all use parallel MCTS on Agent 3 with 5 threads. We conducted another experiment with different order of players.

Method	Win Rate	Average Score
First Player	100%	28.48377
Last Player	100%	26.793

Table 8. The result from parallel MCTS agent being first player and being last player in Game 3

Method	Win Rate	Average Score
First Player (Against together)	80%	29.023035
Last Player (Against together)	20%	29.51985

Table 9. The result from parallel MCTS agent being first player and being last player together in Game 3

The order of players has quite impact on performance. It shows that being the first player seems better.

- Different strategy:

We used another way to simulate legal steps of each state. We assumed that when the sheep of opponent split into two group, the one which is emitted has only one sheep. We conducted 20 games for this session with new way being the last player in first 10 games and being the first player in last 10 games in order to ignore the impact from the order of players.

Method	First 10 games Win Rate	Last 10 games Win Rate
New Way	40%	30%
Old Way	60%	70%

Table 10. The result from parallel MCTS agent being first player and being last player together with different strategy in Game 3

The result shows that our original way performs better than new way as the old way consider more possible legal steps, which can make MCTS make decisions precisely.

Game 4

In this session, we don't conduct experiment about parallel and non-parallel MCTS since based on past 3 Games, it can be inferred that parallel MCTS is better or at least equal to non-parallel MCTS. Thus, we start from the order of players.

- The agent being a first player and being a last player:

In this session, we all use parallel MCTS on Agent 4 with 5 threads to conduct the experiment with different order of players.

Method	Win Rate	Average Score
First Player	100%	23.691
Last Player	100%	22.156

Table 11. The result from parallel MCTS agent being first player and being last player in Game 4

Method	Win Rate	Average Score
First Player (Against together)	80%	22.094
Last Player (Against together)	20%	22.195

Table 12. The result from parallel MCTS agent being first player and being last player together in Game 4

The result still shows that playing with first player in Battle Sheep is much better than with last player.

- Different strategy:

In this session, still, we used another way to evaluate to choose best action. Old way we implement is that make our agent consider its teammate during evaluate value and choose action to do. However, we design a new way to test if it performs better or not, which is let Agent 4 consider itself only just like Agent 1. First, we conduct 20 games for each strategy playing with player 1 against 3 sample players.

Method	Win Rate	Average Score
Agent 4	100%	23.691
Agent 1	100%	20.798

Table 13. The result from parallel MCTS Agent 1 and Agent 4 being first player in Game 4

Then, we additionally consider impact from the order of players and put them to play together for first 10 games with Agent 4 playing as player 1 and Agent 1 playing as player 4 and switch order for last 10 games.

Method	First 10 games Win Rate	Last 10 games Win Rate
Agent 4	60%	50%
Agent 1	40%	50%

Table 14. The result from parallel MCTS Agent 1 and Agent 4 being first player and last player together in Game 4

IV. Discussion

- Based on our experiment, first, we observe that the order of players has large impact on the win rate. The agents which is player 1 usually perform better and stably.
- It is difficult to come up with a suitable strategy on Game 4, which can make agent perform well. We have observed that there are some situations would hinder the teammate's performance resulting in the score of our team decreasing. For instance, if our sheep touch teammate's sheep, our teammate will be restricted because sample player doesn't have ability to know our agent is its teammate.
- The method we choose to implement is MCTS. However, AlphaZero is regarded as much better way. In fact, we only need design several algorithms, including self-play function and training model. In future improvements, we would apply AlphaZero to our implementation to strengthen out agents.

VI. References

- LiaoWC / MCTS (Monte Carlo Tree Search) 演算法,
<https://liaowc.github.io/blog/mcts-monte-carlo-tree-search/>
- David Silver, Julian Schrittwieser, Karen Simonyan / Mastering the Game of Go without Human Knowledge,
https://discovery.ucl.ac.uk/id/eprint/10045895/1/agz_unformatted_nature.pdf

- Fig.1. source: By Rmoss92 - Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=88889583>
- Fig. 2. source:
Guillaume M.J-B. Chaslot, Mark H.M. Winands, and H. Jaap van den Herik /
Parallel Monte-Carlo Tree Search,
<https://dke.maastrichtuniversity.nl/m.winands/documents/multithreadedMCTS2.pdf>

VII. Contribution

- 110550170 戚維凌 – Code: Game 3, Report: Methods & Implementation Details
- 110550034 孫承瑞 – Code: Game 1 & Game 2, Report: Experiments
- 110550175 鄭栩安 – Code: Game 4, Report: Discussion